AI Quality Inspection for Manufactured Parts

Interview manual · PyTorch · TensorFlow/Keras · CAD (CadQuery) · Computer Vision

Karanam Saicharan · github.com/KSC0219

This manual explains the whole project so you can present it confidently: what it does, how every part works, the engineering problems solved while building it, and the questions interviewers ask. Read it once fully, then use the **pitch** (page 2) and the **Q&A** (section 10) before each interview.

> **Before you use this manual**
> Numbers marked **[fill]** come from *your* training run on Colab (results/comparison.md). Never quote a number you did not produce. Everything else in this manual was measured while building the code.

Section	What it gives you
1. Pitch and resume bullets	30-second pitch, 2-minute walkthrough, resume bullets, Naukri text
2. The problem	Why automated inspection matters and why it is hard
3. Architecture	One diagram of the full system and the tech stack
4. CAD stage	Parametric part, defect injection, rendering, STEP files
5. Deep-learning models	Keras MobileNetV2 vs PyTorch ResNet18, training recipe
6. Evaluation	Metrics, the recall-first threshold, the three experiments, your results
7. Grad-CAM and CAD dimension check	Explainability and measuring against drawing tolerance
8. Deployment	TFLite, FastAPI, Streamlit, process isolation
9. Problems I solved	Eight real debugging stories, ready to tell
10. Interview Q&A	30 questions with crisp answers
11. Preparation checklist and glossary	What to do before the interview, key terms

1. Pitch, resume bullets and Naukri text

30-second pitch

> Say this in the interview
> I built an AI system that inspects cast metal parts from photos and flags defects like cracks, blowholes and broken edges. I trained it in both **PyTorch** and **TensorFlow/Keras** with transfer learning and compared them on the same data. The unusual part is the **CAD** side: I modelled the part in CadQuery, created defects on the 3D model, and rendered realistic training images, so the model can learn even when real defect photos are scarce. It also measures the shaft bore against the CAD tolerance, explains each decision with Grad-CAM, and is deployed as a REST API, a web demo and a TFLite model for edge devices.

2-minute walkthrough (follow this order)

1. **Problem.** Manual visual inspection of castings is slow and inconsistent; a missed defect reaches the customer. Real defect photos are also scarce, because most parts are good.
2. **Data.** A public Kaggle dataset of pump-impeller photos (OK vs defective), plus synthetic images I generated from a CAD model.
3. **CAD stage.** A parametric impeller in CadQuery (100 mm disc, 7 curved vanes, 12 mm bore). Defects are boolean operations on the solid: subtract spheres for blowholes, thin slots for cracks, cylinders for edge chips; add a thin plate for flash. A renderer turns each model into photos with random light, texture, background and camera pose (domain randomisation).
4. **Models.** MobileNetV2 in Keras and ResNet18 in PyTorch, both ImageNet transfer learning in two phases: train the new head, then fine-tune the top layers. Identical data split and metrics for a fair comparison.
5. **Decision.** I tune the threshold on validation data so at least 98% of defective parts are caught, because a missed defect costs much more than a false alarm.
6. **Experiments.** Real photos only, CAD images only, and CAD images plus 10% of the real photos, all tested on the same real test set. Result: [fill: your key finding, e.g. 'CAD + 10% real reached X% recall, close to the Y% of full real data'].
7. **Trust and deployment.** Grad-CAM heatmaps show where the model looked. A dimension check measures the bore against 12 ± 0.5 mm. Served with FastAPI and Streamlit, and exported to TFLite (about 3.8x smaller). 25 automated tests.

Resume bullet points

Use 3–4 bullets. Replace [fill] with your Colab numbers; until then use the version without numbers.

- Built an AI visual-inspection system for cast impellers in **PyTorch** (ResNet18) and **TensorFlow/Keras** (MobileNetV2) using transfer learning; reached [fill]% defect recall and [fill]% accuracy on real test photos.
- Generated synthetic training data from a parametric **CAD** model (CadQuery): injected 5 casting-defect types via boolean operations and rendered photo-realistic images with domain randomisation; CAD + 10% real images reached [fill]% recall.
- Tuned the decision threshold for ≥98% defect recall, explained predictions with **Grad-CAM**, and added a CAD tolerance check measuring bore diameter to 0.22 mm median error.
- Deployed via **FastAPI** REST API, **Streamlit** demo and a quantized **TFLite** model (3.8x smaller, identical decisions); 25 automated tests; process isolation for TensorFlow and PyTorch.

Version without numbers (use before training): “Built an end-to-end AI quality-inspection pipeline in PyTorch and TensorFlow/Keras with CAD-generated synthetic training data (CadQuery), Grad-CAM explanations, CAD tolerance checking, and FastAPI/Streamlit/TFLite deployment.”

Naukri project entry

Field	What to enter
Project title	AI Quality Inspection for Manufactured Parts (CAD + Deep Learning)
Client	Personal Project
Status	Finished
Details of project	AI system that detects casting defects (cracks, blowholes, edge chips, flash) in photos of pump impellers. Trained and compared PyTorch (ResNet18) and TensorFlow/Keras (MobileNetV2) with transfer learning. Built a parametric CAD model in CadQuery, injected defects with boolean operations and rendered synthetic training images with domain randomisation. Threshold tuned for 98% defect recall, Grad-CAM explanations, bore-diameter check against CAD tolerance, TFLite export for edge, FastAPI + Streamlit deployment. Results: [fill].
Role	Developer (individual project)
Skills used	Python, PyTorch, TensorFlow, Keras, CNN, Transfer Learning, Computer Vision, OpenCV, CAD, CadQuery, Grad-CAM, TFLite, FastAPI, Streamlit, Scikit-learn

2. The problem

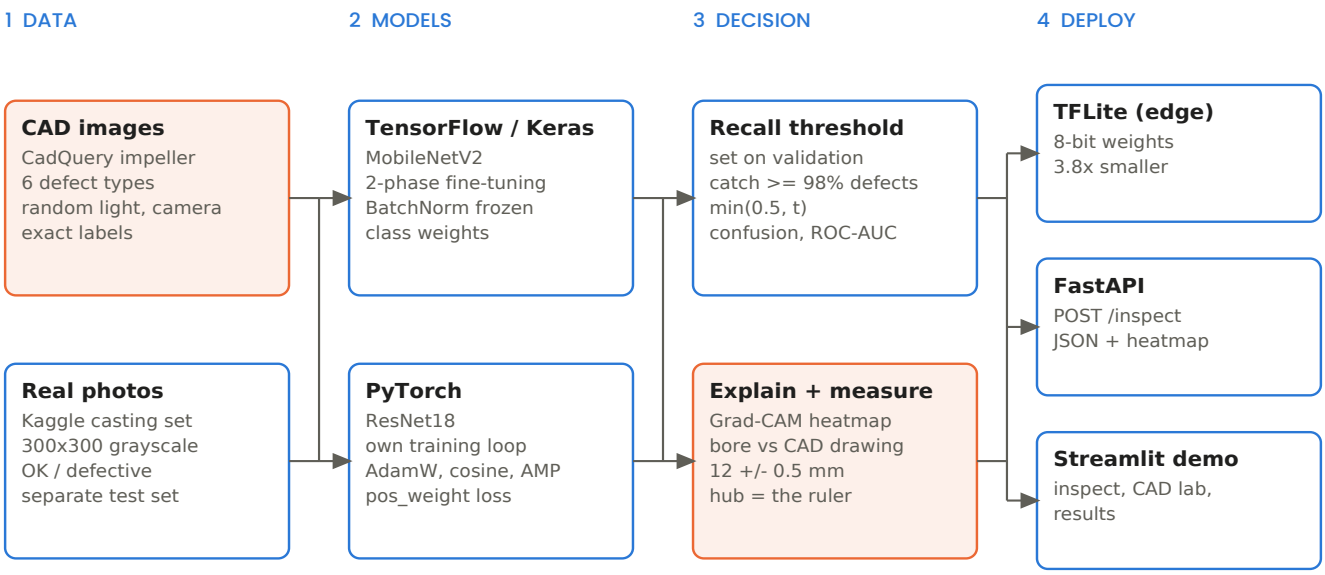
Foundries and machine shops inspect parts visually. Human inspection is slow, gets tired, and two inspectors can disagree. The cost of errors is not symmetric: a **missed defect** (a cracked part shipped) causes returns, failures in the field and lost customers, while a **false alarm** only costs a manual re-check.

Two practical difficulties make this a good AI project:

- **Defects are small** relative to the part: a pinhole may be a few pixels. The model must look at fine detail, not the overall shape.
- **Defect photos are scarce.** Most production parts are good, and a new product has no defect history at all. This is why the project generates defect images from CAD.

> Say this in the interview
> “The key metric is not accuracy, it is how many defective parts we catch, called recall. So I tuned the decision threshold on validation data to catch at least 98% of defects, accepting a few more false alarms.”

3. Architecture and tech stack



Data flows left to right. Orange boxes are the CAD parts of the project; blue boxes are standard deep-learning parts. Both frameworks get the same data split and the same metrics.

Layer	Technology	Why this choice
CAD	CadQuery (OpenCASCADE kernel), STEP export	Free, scriptable in Python, same B-rep kernel family as commercial CAD; STEP opens in SolidWorks, CATIA, NX, FreeCAD
Rendering	NumPy z-buffer + OpenCV	No GPU/OpenGL needed, runs on Colab and laptops; full control of lighting randomisation
Model 1	TensorFlow 2 + Keras 3, MobileNetV2	Small (≈2.3M params), built for phones: good for edge deployment and TFLite
Model 2	PyTorch + torchvision, ResNet18	Strong, standard baseline (≈11M params); shows a hand-written training loop
Metrics	scikit-learn	Precision, recall, F1, ROC-AUC, confusion matrix
Explainability	Grad-CAM (both frameworks)	Shows where the model looked; builds trust with quality engineers
Measurement	OpenCV Hough + least-squares circle fit	Sub-pixel bore measurement against the CAD drawing value
Serving	FastAPI, Streamlit, TFLite	REST for factory systems, demo UI for people, TFLite for edge cameras
Quality	pytest (25 tests)	CAD, data, metrics, both models and the API are tested

4. CAD stage: synthetic data and dimensions

4.1 The parametric part

The impeller is built in **qi/cad/part.py** from a *specification* of design dimensions, exactly like a parametric CAD model where changing a number rebuilds the part:

Parameter	Value	Note
Disc diameter / thickness	100 mm / 6 mm	top edge chamfered 0.6 mm (cast parts have broken edges)
Hub diameter / height	30 mm / 10 mm	the hub later serves as the measurement ruler
Shaft bore	12 mm ± 0.5 mm	the toleranced feature that is measured
Vanes	7 curved vanes, 3 mm thick, 5 mm high	each sweeps 55° from hub to rim
Manufacturing variation	±0.3 mm on each dimension	every generated part is slightly different, like real castings

Solid modelling steps: extrude a circle (disc) → chamfer top edge → union a cylinder (hub) → union 7 extruded curved polygons (vanes) → cut a cylinder (bore). The result is an exact **B-rep** solid (boundary representation: faces, edges and vertices with exact geometry), which is what professional CAD tools use.

4.2 Defects as boolean operations

Defect	How it is made on the CAD solid	Real-world cause
Blowhole	subtract 1–3 spheres, radius 1.2–3 mm, at the surface	gas trapped during casting
Pinholes	subtract a cluster of 6–13 tiny spheres (0.6–1.1 mm)	fine gas porosity
Crack	subtract a zig-zag of 2–4 thin slots (0.8–1.4 mm wide, 2.2–3.5 mm deep)	hot tear while metal cools
Edge chip	subtract a cylinder (radius 3–7 mm) overlapping the rim	damage in handling or fettling
Flash / burr	add a thin irregular plate (0.6–1.4 mm) at the rim	metal leaking into the mould joint
Bore oversize	rebuild with a bore 1.2–3 mm too large	machining error (dimensional, not visual)

Because the defect is created on purpose, every image comes with **exact ground truth**: defect type, position and size are saved in *metadata.csv*. Real datasets rarely have this.

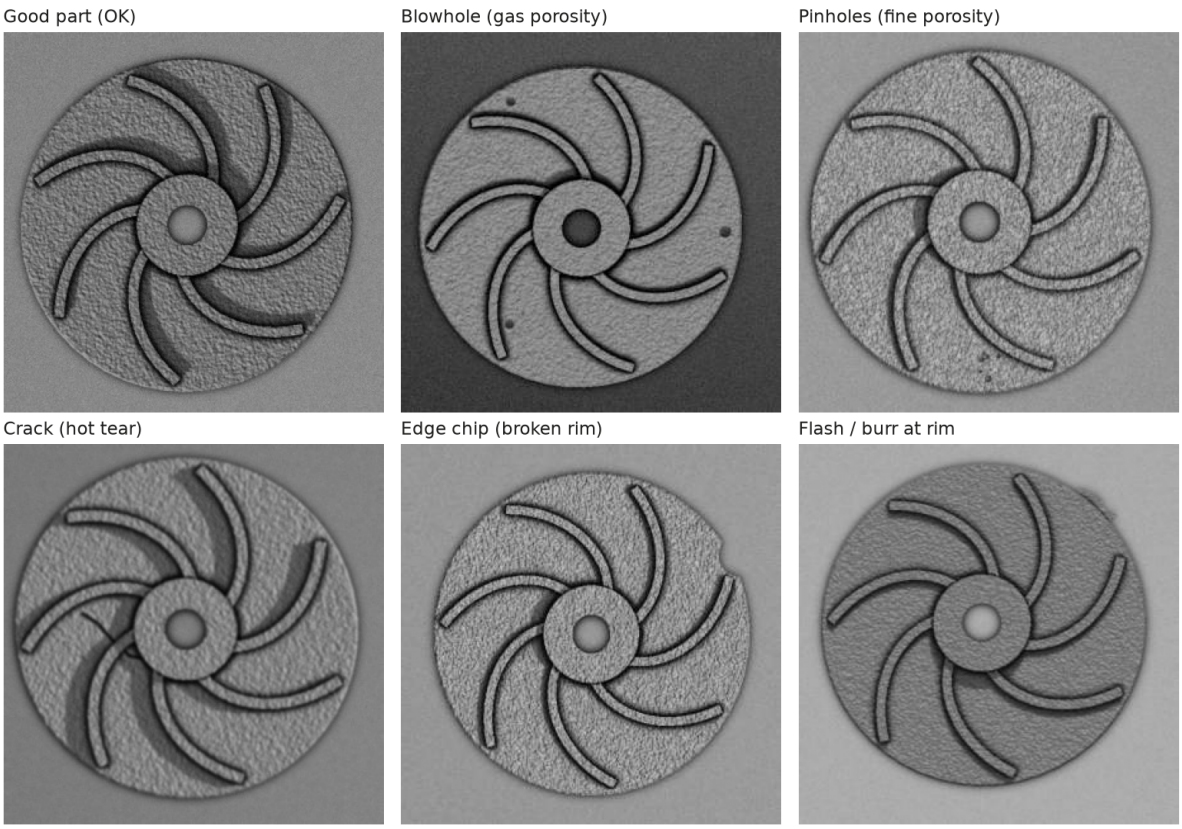


Figure 1. CAD-generated photos: a good part and the five visual defect types (each image is a different random part, light and camera).

4.3 From CAD model to photo

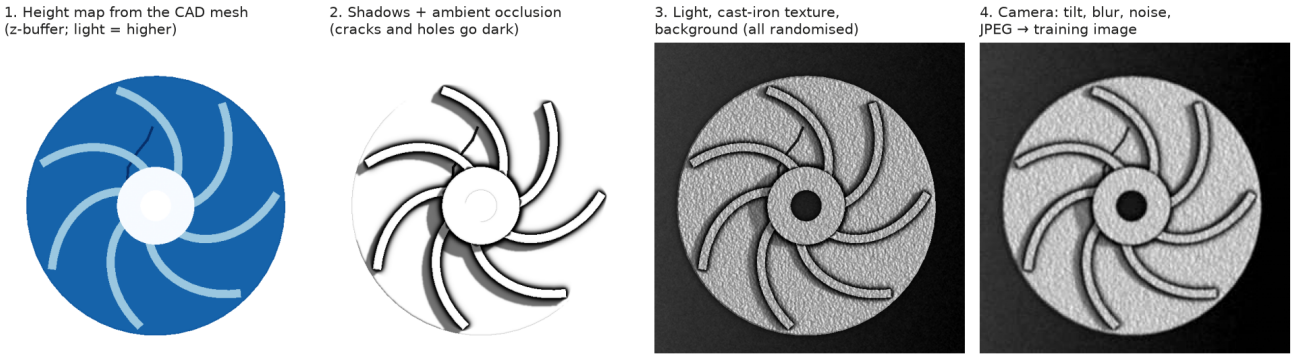


Figure 2. Rendering pipeline for one cracked part (qi/cad/render.py).

1. **Tessellate** the exact solid into triangles (a mesh), then **rasterize** it from above with a z-buffer: for each pixel keep the highest surface. Output: a height map and surface normals. The part is rotated, scaled and shifted first, so every view is a different pose.
2. **Shadows and ambient occlusion** are computed from the height map: points lower than their surroundings (cracks, holes, the bore) receive less light and look dark, as in a real photo.
3. **Lighting and material:** random light direction and height, matte cast-iron albedo with fractal noise, a bump-mapped rough surface, weak specular highlights, random background brightness and gradient.
4. **Camera:** slight perspective tilt, blur, exposure change, sensor noise and JPEG compression, saved as 300×300 grayscale like the real dataset. About 0.2 s per image; parts are built in parallel processes.

> Say this in the interview
> “Randomising light, texture, background and camera for every image is called **domain randomisation**. If the model has seen thousands of variations, a real photo looks like just another variation. That is how you reduce the **sim-to-real gap**.”

STEP files: the generator also exports sample_part_ok.step and one per defect. Open them in FreeCAD (free) before the interview so you can show the 3D model and the defect geometry.

5. Deep-learning models

	TensorFlow / Keras	PyTorch
Backbone	MobileNetV2, ImageNet weights	ResNet18, ImageNet weights
Parameters	≈2.3 million	≈11.2 million
Head	GlobalAveragePooling → Dropout 0.3 → Dense(1, sigmoid)	Dropout 0.3 → Linear(512, 1) → logit
Phase 1 (5 epochs, lr 1e-3)	backbone frozen, only head trains	only fc (head) trains
Phase 2 (10 epochs, lr 1e-4)	top 40 layers unfrozen, BatchNorm kept frozen	layer3 + layer4 + head, AdamW, cosine schedule
Loss and imbalance	binary cross-entropy + class weights	BCEWithLogitsLoss with pos_weight
Augmentation	flips, rotation ±180°, zoom 10%, contrast 20%, brightness 15% (Keras layers, GPU)	same set with torchvision transforms
Training loop	model.fit with callbacks	hand-written loop, mixed precision (AMP) on GPU
Stopping	EarlyStopping on val loss, best weights restored	same logic written by hand
Grad-CAM	tf.GradientTape on backbone output	forward and backward hooks on layer4

Why two-phase transfer learning

ImageNet features (edges, textures, shapes) are already useful for defects. If you train all layers at once with a high learning rate, the large random gradients from the new head damage those features. So first the new head learns on frozen features; then the top of the backbone is fine-tuned with a 10× smaller learning rate.

Why BatchNorm stays frozen in Keras fine-tuning

BatchNorm layers store running mean and variance from ImageNet. Fine-tuning with small batches would replace them with noisy statistics. The backbone is therefore always called in inference mode (training=False). In PyTorch with batch size 32 the update is stable, so the standard behaviour is kept. This difference is a good interview detail.

Fair comparison

Both frameworks receive the **same file lists**: one stratified split with a fixed seed, built in qi/data.py. All photos of one CAD part stay on the same side of the split, so validation never contains a near-copy of a training image (this would be **data leakage** and give falsely high scores). One shared routine (qi/runner.py) trains, evaluates and writes the same reports for both.

6. Evaluation

6.1 Metrics in plain words

Metric	Meaning here	Formula
Recall (defective)	share of bad parts the model catches	TP / (TP + FN)
Precision (defective)	of the parts flagged bad, share really bad	TP / (TP + FP)
Accuracy	share of all parts classified correctly	(TP + TN) / all
F1	balance of precision and recall	2PR / (P + R)
ROC-AUC	how well scores separate bad from good, at every threshold	area under ROC curve
Missed defects	false negatives: the costly mistake	FN
False alarms	good parts sent for re-check	FP

6.2 The recall-first threshold

The model outputs a probability that the part is defective. Instead of always using 0.5, the threshold is chosen on the validation set as **min(0.5, t)**, where t is the highest threshold that still catches at least 98% of validation defects. The threshold can only move *down* from 0.5 (flag more parts), never up. The cap was added after a real bug: on a perfectly separated validation set, t became 0.998 and the model missed defects on new data (story 6 in section 9).

6.3 The three experiments

Experiment	Training data	Question it answers
real	real photos only	baseline: how good can it get with plenty of real data?
cad_only	CAD images only, zero real photos	can a model trained purely from CAD work on real parts? (sim-to-real gap)
cad_plus_10	CAD images + 10% of the real photos	the practical case: a few real photos plus CAD data

All six models (3 experiments × 2 frameworks) are tested on the **same real test photos**.

6.4 Your results (fill in from results/comparison.md)

Training data	Framework	Accuracy	Defect recall	Missed	False alarms
Real photos	Keras	[fill]	[fill]	[fill]	[fill]
Real photos	PyTorch	[fill]	[fill]	[fill]	[fill]
CAD only	Keras	[fill]	[fill]	[fill]	[fill]
CAD only	PyTorch	[fill]	[fill]	[fill]	[fill]
CAD + 10% real	Keras	[fill]	[fill]	[fill]	[fill]
CAD + 10% real	PyTorch	[fill]	[fill]	[fill]	[fill]

6.5 How to explain whatever results you get

- **CAD only is much lower than real:** expected. That is the sim-to-real gap: my renderer is simpler than reality (real lighting, real surface finish, real defect shapes). Then say what helps: more realistic materials, more randomisation, or a few real images.
- **CAD + 10% real is close to real:** this is the strongest story. With only a tenth of the real photos, CAD data recovers most of the performance, so a factory needs far fewer real defect photos.
- **Keras and PyTorch are close:** normal. The architecture and data matter more than the framework. Point out the trade-off instead: MobileNetV2 is about 5× smaller and faster, ResNet18 is a little stronger.
- **One framework clearly wins:** report it honestly and give a reason (model capacity, fine-tuning depth, augmentation).

7. Grad-CAM and the CAD dimension check

7.1 Grad-CAM in four steps

1. Run the image through the network and keep the feature maps of the last convolution block (7×7 grid, many channels).
2. Compute the gradient of the 'defective' output with respect to each feature map.
3. Average each gradient map to one weight per channel: how important that channel is for 'defective'.
4. Weighted sum of the feature maps, keep positive values, normalise to 0–1, resize onto the image (orange overlay).

If the heatmap lights up on the crack or blowhole, the model learned the right thing. If it lights up on the background, the model learned a shortcut, which is a reason to collect better data. Grad-CAM examples of caught defects, missed defects and false alarms are saved for every model (gradcam_examples.png).

7.2 Measuring the bore against CAD tolerance

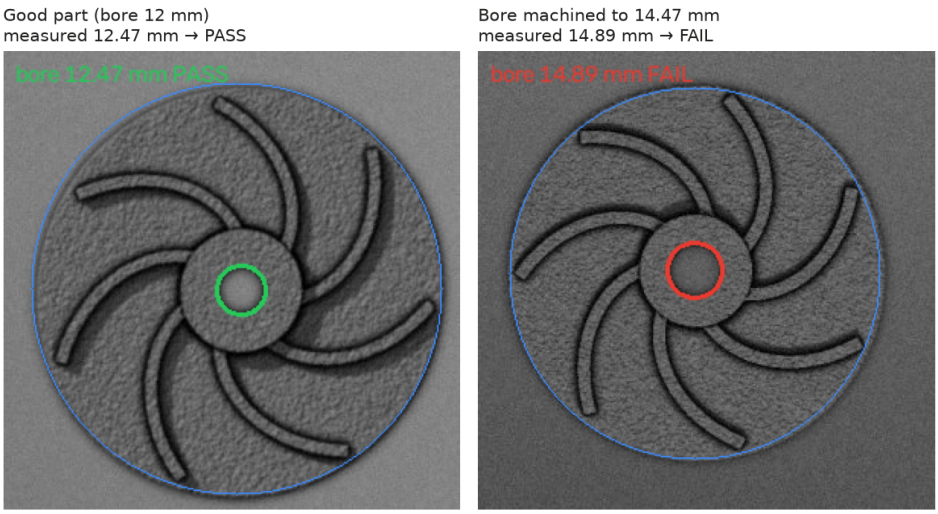


Figure 3. Dimension check on CAD-generated parts: blue = detected rim, green/red = fitted bore circle.

1. Find the part's rim with the Hough circle transform, then refine it.
2. Find the hub circle. Its CAD diameter is 30 mm, so **mm per pixel = 30 / hub diameter in pixels**. The part itself is the ruler, so no separate camera calibration is needed, and hub and bore are at the same height (no perspective difference).

3. Find the bore: cast 90 rays from the centre, take the strongest brightness edge on each ray, fit a circle by least squares and remove outliers (vane roots, defects). This gives sub-pixel accuracy.
4. Bore in mm vs 12 ± 0.5 mm → PASS or FAIL. If the edge contrast is too low the answer is UNCERTAIN instead of a guess.

Measured on 100 CAD-generated parts (known true size)	Result
Median absolute error	0.22 mm
Parts measured within ±0.5 mm	90% (88 of 98)
Good parts passed	46 of 50 (1 uncertain)
Oversize bores caught (FAIL)	45 of 50 (1 uncertain)
Image resolution	300×300 px ≈ 0.4 mm per pixel

Be precise in interviews: this was validated on CAD-generated images where the true size is known. On real photos it needs a calibrated camera setup and the real drawing values. Accuracy scales with resolution.

8. Deployment

Component	What it does	How to run
TFLite export	Keras → TensorFlow Lite with dynamic-range quantization (8-bit weights). In testing: 9.65 MB → 2.53 MB (3.8× smaller), identical decisions on 30/30 test images, max probability difference 0.0016.	python scripts/export_tflite.py
FastAPI	GET /health, POST /inspect: upload an image, choose keras / pytorch / tflite, get label, probability, threshold, Grad-CAM PNG (base64) and optional bore check. Clear 400 errors for bad files.	uvicorn api.main:app → /docs
Streamlit app	Inspect a photo with any model, CAD defect lab (build a part with a chosen defect, render it, download STEP, run the model), results dashboard.	streamlit run app/streamlit_app.py
Process isolation	Each framework's model runs in its own worker process; the API process never imports TensorFlow or PyTorch.	automatic

> Say this in the interview
> “TensorFlow and PyTorch crashed with a segmentation fault when loaded into one Python process, because their native libraries clash. So I run each training job and each model behind the API in its own process, which also means one crashing model cannot take down the server.”

9. Problems I solved while building it

Interviewers love “tell me about a hard problem.” These eight happened during development. Use the **problem → cause → fix → result** structure.

1. Ghost parts in the corners of rendered images	
Cause	Rotating the finished image filled the corners by mirroring, which created half-impellers at the edges.
Fix	Rotate the 3D model before rendering instead of the image.
Result	Clean images; the model cannot learn a fake corner pattern.

2. The bore was invisible in some renders	
Cause	The background seen through the hole had the same brightness as the part.
Fix	Physically-motivated darkening: deep narrow holes get little light (occlusion from the height map).
Result	Bore always visible, images more realistic; bore measurement became possible.

3. Bore measurement off by up to 2 mm	
Cause	Hough circles are only pixel-accurate and confused the rim with the vane tips.
Fix	Ray-based edge detection + least-squares circle fit with outlier removal, and the hub as the ruler.
Result	Median error 0.22 mm; 90% within ±0.5 mm.

4. TensorFlow + PyTorch segmentation fault	
Cause	Both frameworks loaded into one process; their native libraries clash (crashed in either import order).
Fix	Process isolation for training jobs and for the API/app models.
Result	Full 6-model experiment grid and API run reliably.

5. Parallel data generation froze	
Cause	Worker processes were forked after the CAD kernel (OpenCASCADE) was loaded: a classic fork deadlock.
Fix	Use the 'spawn' start method so workers start fresh.
Result	Parallel generation works every time.

6. Threshold overfitted to validation	
Cause	Validation was perfectly separated, so the 98%-recall threshold became 0.998; on test, recall dropped to 67%.
Fix	Cap the threshold at 0.5: it may only move down to catch more defects.
Result	Robust behaviour on new data; covered by a unit test.

7. TFLite conversion failed	
Cause	The random-augmentation layers were exported into the graph and TFLite cannot convert those ops.
Fix	Export an inference-only model that reuses the trained layers without augmentation.
Result	2.53 MB TFLite model, same decisions as Keras.

8. Keras model would not learn from scratch	
Cause	Keras BatchNorm updates running statistics slowly (momentum 0.99), so validation used wrong statistics.
Fix	For scratch training set momentum to 0.9 (pretrained runs keep BatchNorm frozen anyway).
Result	Sanity test reached 100% train and validation accuracy.

10. Interview questions and answers

Deep learning basics

What is transfer learning and why use it here?

Reusing a network trained on a large dataset (ImageNet, 1.2 million images) and adapting it to a new task. Early layers detect edges and textures that are useful for defects too, so we need far less data and training time than training from scratch.

Why MobileNetV2 in Keras and ResNet18 in PyTorch?

MobileNetV2 uses depthwise-separable convolutions, so it is small (≈2.3M parameters) and fast: ideal for TFLite on edge devices. ResNet18 (≈11M) is a strong standard baseline with residual connections. Comparing them shows the size/speed vs accuracy trade-off.

Why a sigmoid with one output instead of softmax with two?
It is a binary problem: one probability P(defective) is enough, and it makes the threshold easy to tune. Softmax over two classes is mathematically equivalent.

Why BCEWithLogitsLoss in PyTorch instead of sigmoid + BCE?
It combines both in one numerically stable operation (log-sum-exp trick) and accepts pos_weight for class imbalance.

How did you handle class imbalance?
Class weights in Keras and pos_weight in PyTorch, so the rarer class counts more in the loss. Plus the recall-based threshold, and stratified splits that keep the class ratio equal in train and validation.

What data augmentation did you use and why?
Flips and full rotation (an impeller looks the same at any angle), zoom, contrast and brightness. It teaches the model that position and lighting do not matter, only the defect, and reduces overfitting.

How did you prevent overfitting?
Transfer learning, augmentation, dropout 0.3, a separate validation set, early stopping with the best weights restored, and a small fine-tuning learning rate.

Adam vs AdamW? Why a cosine schedule?
AdamW applies weight decay correctly (decoupled from the gradient update), which regularises better. A cosine schedule lowers the learning rate smoothly so fine-tuning settles into a good minimum.

What is mixed precision (AMP)?
Computing in 16-bit floats where it is safe and 32-bit where needed. On a GPU it is faster and uses less memory; a gradient scaler prevents tiny gradients from becoming zero.

Metrics and decisions

Why not just report accuracy?
If 60% of parts are defective, a model that always says 'defective' gets 60% accuracy and is useless. Accuracy hides which mistakes happen. In inspection a missed defect is the expensive mistake, so recall on defects is the key metric.

How did you pick the threshold?
On validation data: the highest threshold that still catches at least 98% of defects, capped at 0.5 so it can only move down. It is chosen on validation, never on test, so the test score stays honest.

What does ROC-AUC tell you?
How well the scores separate defective from good parts across all thresholds. 1.0 = perfect, 0.5 = random. It is independent of the threshold choice.

What is data leakage and how did you avoid it?
When information from the evaluation data gets into training, giving falsely high scores. I split the CAD images by physical part, so all photos of one part are on the same side, and I never tune anything on the test set.

CAD and synthetic data

What is CAD and what is a B-rep?
Computer-aided design: creating exact 3D models of parts. A B-rep (boundary representation) describes a solid by its exact faces, edges and vertices, unlike a mesh, which approximates surfaces with triangles. I model in B-rep and tessellate to a mesh only for rendering.

What is a parametric model?

A model driven by named dimensions (diameter, vane count, bore). Change a number and the part rebuilds. I use this to create variation between parts and to know the exact nominal dimensions for measurement.

What are boolean operations?

Union, subtract and intersect of solids. A blowhole is the part minus a sphere, a crack is the part minus thin slots, flash is the part plus a thin plate.

What is a STEP file?

The standard neutral CAD exchange format (ISO 10303). It opens in SolidWorks, CATIA, Siemens NX and FreeCAD.

What is the sim-to-real gap and how do you reduce it?

Models trained on synthetic images often do worse on real photos because the images differ in subtle ways. I reduce it with domain randomisation (random light, texture, background, camera, noise) and by mixing in a small share of real images.

Why generate synthetic data at all if a real dataset exists?

For a new product there is no defect history, and rare defect types have very few photos. CAD gives unlimited, perfectly labelled examples of any defect, at any size and position, before production even starts.

Explainability, measurement and deployment

How does Grad-CAM work?

It weights the last convolutional feature maps by how much they increase the 'defective' score (averaged gradients), sums them and keeps the positive part. The result is a heatmap of the evidence the model used.

How accurate is your measurement and what limits it?

Median error 0.22 mm on CAD-generated parts at 300×300 pixels (about 0.4 mm per pixel). Resolution, lighting contrast and camera tilt limit it. A higher-resolution, calibrated camera would improve it proportionally.

What is quantization?

Storing weights as 8-bit integers instead of 32-bit floats. The model becomes about 4× smaller and faster on edge hardware, with a very small accuracy loss. I checked that TFLite and Keras gave the same decisions.

How would you deploy this in a real factory?

A fixed camera with controlled lighting over the conveyor, the TFLite model on an edge device or the API on a local server, a reject signal to the PLC when a part is flagged, logging of images and decisions, and periodic retraining with new images reviewed by quality engineers.

Keras vs PyTorch: which do you prefer?

Keras is faster to write (fit, callbacks) and has the smoothest path to TFLite. PyTorch gives full control of the training loop and is more common in research. I can work in both; for a phone or edge product I would pick Keras/TFLite.

About the project itself

What was the hardest part?

Making synthetic images realistic enough and measuring the bore accurately. Pick one story from section 9 and tell it as problem, cause, fix, result.

What are the limitations?

One part design and a simplified renderer; the measurement is validated on synthetic images; the real dataset has one lighting setup. A new product or lighting needs new data or fine-tuning.

What would you do with more time?

Defect localisation with a detector or segmentation model (bounding boxes, not just OK/defective), anomaly detection trained only on good parts, a physically-based renderer (Blender) for better realism, and active learning to label the most useful real images first.

How did you test the code?

25 pytest tests: CAD geometry (each defect changes the volume in the right direction), rendering, measurement pass/fail, metrics, data splits without leakage, both models (train, predict, Grad-CAM, save/load, TFLite agreement) and the API end to end.

Did you use AI tools to build it?

Answer honestly. A good answer: “I used AI assistance for parts of the code, like most developers do now, but I ran every experiment, debugged the problems, and I can explain every design decision.” Then prove it by explaining one file in depth.

11. Preparation checklist and glossary

Before the interview

- Run the Colab notebook end to end and fill every [fill] in this manual from results/comparison.md.
- Open sample_part_crack.step in FreeCAD and be ready to show it.
- Open results/real/*/gradcam_examples.png: explain one caught defect and one mistake.
- Read these files once: qi/cad/part.py, qi/cad/render.py, qi/metrics.py, qi/keras_model.py, qi/torch_model.py.
- Push to GitHub (README already written) and deploy the Streamlit app; put both links on your resume.
- Practise the 30-second pitch aloud until it takes under 40 seconds.

What you can and cannot claim

You can say	Do not say
Built and tested end to end; trained on a public industrial dataset	Deployed in a factory or used in production
CAD-generated synthetic data with exact labels	That synthetic data alone matches real data, unless your numbers show it
Bore measurement: 0.22 mm median error on CAD-generated parts	That it measures real parts to 0.22 mm
Your own Colab numbers	Any number you did not produce

Commands

```
python scripts/download_data.py          # real dataset (Kaggle)
python scripts/generate_cad_data.py      # CAD synthetic images + STEP files
python scripts/run_experiments.py        # 3 experiments x 2 frameworks
python scripts/export_tflite.py          # edge model
python scripts/compare.py                # comparison table + chart
python scripts/measure_check.py          # bore measurement accuracy
uvicorn api.main:app --reload            # REST API, docs at /docs
streamlit run app/streamlit_app.py       # demo app
python scripts/run_tests.py              # 25 tests
```

Glossary

Term	Meaning
Ambient occlusion	darkening of points that are hard for light to reach (cavities, cracks)
B-rep	boundary representation: exact CAD solid made of faces, edges and vertices
Domain randomisation	randomising rendering conditions so a model generalises to real images
Fine-tuning	continuing training of pretrained layers with a small learning rate
Grad-CAM	gradient-weighted class activation map: heatmap of model evidence
Recall / sensitivity	share of actual defects that are detected
Sim-to-real gap	performance drop when a model trained on synthetic data meets real data
Tessellation	converting exact CAD surfaces into a triangle mesh
Threshold	probability above which a part is called defective
TFLite / LiteRT	TensorFlow's runtime for phones and edge devices
Z-buffer	rendering method that keeps the nearest surface for every pixel